
Documentation Technique Complète

Détection de Contours, Lissage
& Suppression d'Arrière-plan — v3

Interface Streamlit · Python · Analyse Numérique

Projet d'Analyse Numérique

Traitement numérique d'images

11 mai 2026

Cette documentation décrit en détail une application complète d'analyse numérique d'images implémentée en Python avec une interface Streamlit. Elle couvre les fondements mathématiques, les bibliothèques utilisées, les algorithmes de lissage, de détection de contours, de segmentation et de composition avancée (équation de Poisson), ainsi que l'architecture générale du projet.

Table des matières

1	Vue d'ensemble	5
2	Architecture du projet	6
3	Bibliothèques et dépendances	8
3.1	Bibliothèques obligatoires	8
3.2	Bibliothèque optionnelle	8
3.3	Modules Python standard	9
3.4	Sous-modules <code>scipy</code> utilisés	9
4	Fondements mathématiques	10
4.1	Représentation numérique d'une image	10
4.2	Convolution discrète 2D	10
4.3	Filtre Gaussien	10
4.4	CLAHE	10
4.5	Filtre bilatéral	11
4.6	Filtre médian	11
4.7	Interpolation par splines bicubiques	12
4.8	Opérateur de Sobel et Non-Maximum Suppression	12
4.9	Algorithme de Canny	12
4.10	Laplacien du Gaussien (LoG) – Zéro-croisements	13
4.11	Transformée de Fourier 2D et filtres de Butterworth	13
4.12	Saillance spectrale (Hou & Zhang, 2007)	13
4.13	GrabCut	14
4.14	Segmentation K-means en espace LAB	14
4.15	Flood Fill depuis les bords	14
4.16	Alpha matting par distance transform	15
4.17	Transfert de couleur (Reinhard et al., 2001)	15
4.18	Incrustation de Poisson (Pérez et al., 2003)	15
4.18.1	Formulation variationnelle	15
4.18.2	Laplacien discret	16
4.18.3	Trois modes de fusion	16
4.18.4	Boîte englobante après rotation	16
5	Module Pré-traitement & Lissage	17
5.1	<code>normalize</code> et <code>to_uint8</code>	17

5.2	clahe_enhance	17
5.3	gaussian_smoothing	17
5.4	bilateral_smoothing	17
5.5	median_smoothing	17
5.6	spline_smooth	17
6	Module Détection de Contours	18
6.1	sobel_nms	18
6.2	canny_edges	18
6.3	log_zero_crossing	18
6.4	spline_edges	18
6.5	fft_hp et fft_lp	18
6.6	fft_spectrum	18
7	Module Suppression d'Arrière-plan	19
7.1	spectral_saliency	19
7.2	bg_remove_grabcut	19
7.3	bg_remove_border_flood	19
7.4	bg_remove_kmeans	19
7.5	bg_remove_rembg	19
8	Module Raffinement du Masque	20
8.1	fill_holes	20
8.2	remove_islands	20
8.3	edge_snap	20
8.4	alpha_matting	20
8.5	full_refinement	20
9	Module Composition & Poisson Blending	21
9.1	get_poisson_operators	21
9.2	make_S_sparse	21
9.3	poisson_blend_patch	21
9.4	apply_reflection	21
10	Fonctions Utilitaires & Post-traitement	22
10.1	color_transfer	22
10.2	apply_mask_soft	22
10.3	apply_image_adjustments	22
10.4	apply_artistic_mode	22
10.5	generate_shadow	22
10.6	apply_watermark	22

11 Interface Streamlit	23
11.1 Mise en cache	23
11.2 Organisation en onglets	23
11.3 Traitement par lot	23
12 Flux de données (Pipeline)	24
13 Tableau récapitulatif des méthodes	26
Références bibliographiques	27

Chapitre 1

Vue d'ensemble

Cette application constitue un **outil complet d'analyse numérique d'images** qui implémente, depuis zéro ou via des bibliothèques spécialisées, les grandes familles d'algorithmes du traitement d'image :

- ▶ **Lissage et amélioration** — préparation du signal image ;
- ▶ **Détection de contours** — dérivées discrètes, transformée de Fourier ;
- ▶ **Segmentation et suppression de fond** — algorithmes classiques et IA ;
- ▶ **Composition avancée** — résolution d'équations de Poisson ;
- ▶ **Post-traitement visuel** — filigrane, retouche, effets artistiques.

L'application est organisée en **onglets fonctionnels** : Vue unique, Vue complète, Comparaison, Suppression de fond, Composition Poisson, et Avant/Après.

Chapitre 2

Architecture du projet

Structure modulaire de code.py

```
code.py
|
+-- UTILITAIRES                                normalize(), to_uint8()
|
+-- PRE-TRAITEMENT & LISSAGE
|   +-- clahe_enhance()                        Egalisation adaptative d'
|       histogramme
|   +-- gaussian_smoothing() Filtre gaussien (convolution)
|   +-- bilateral_smoothing() Filtre bilateral (preserve
|       les bords)
|   +-- median_smoothing() Filtre median (bruit
|       impulsif)
|   +-- spline_smooth() Lissage par splines
|       bicubiques
|
+-- DETECTION DE CONTOURS
|   +-- sobel_nms() Sobel + Non-Maximum
|       Suppression
|   +-- canny_edges() Algorithme de Canny complet
|   +-- log_zero_crossing() Laplacien du Gaussien (LoG)
|   +-- spline_edges() Gradient sur image splinee
|   +-- fft_hp() Filtre passe-haut FFT (
|       Butterworth)
|   +-- fft_lp() Filtre passe-bas FFT (
|       Butterworth)
|   +-- fft_spectrum() Spectre d'amplitude FFT
|
+-- SUPPRESSION D'ARRIERE-PLAN
|   +-- spectral_saliency() Saillance (Hou & Zhang 2007)
|   +-- bg_remove_grabcut() GrabCut + saillance spectrale
|   +-- bg_remove_border_flood() Flood Fill depuis les
```

```
bords
|   +-- bg_remove_kmeans()    K-means LAB + position
|   +-- bg_remove_rembg()    Reseau U2-Net (deep learning)
|
+-- RAFFINEMENT DU MASQUE
|   +-- fill_holes()          Remplissage (composantes
    connexes)
|   +-- remove_islands()      Suppression des petites
    regions
|   +-- edge_snap()           Alignement sur contours Canny
|   +-- alpha_matting()       Transitions douces (dist.
    transform)
|   +-- full_refinement()     Pipeline complet de
    raffinement
|
+-- APPLICATION DU MASQUE
|   +-- apply_mask_soft()     Composition avec fond colore
|   +-- to_rgba()             Conversion PNG avec
    transparence
|   +-- color_transfer()      Transfert couleur (Reinhard
    2001)
|   +-- apply_reflection()     Effet reflet miroir degrade
|   +-- generate_shadow()      Ombre portee
|   +-- apply_watermark()     Filigrane avec contour
|
+-- POISSON BLENDING
|   +-- get_poisson_operators() Operateurs Laplaciens
    creux
|   +-- make_S_sparse()       Matrice de selection de
    region
|   +-- poisson_blend_patch() Resolution systeme lineaire
    creux
|
+-- INTERFACE STREAMLIT
    +-- cached_contours()     Contours (mis en cache)
    +-- cached_bg()           Segmentation (mise en cache)
    +-- Onglets de visualisation
```


Chapitre 3

Bibliothèques et dépendances

3.1 Bibliothèques obligatoires

TABLE 3.1 – Dépendances principales du projet

Bibliothèque	Version min.	Rôle dans le projet
streamlit	≥ 1.30	Interface web interactive, widgets, onglets, mise en cache
numpy	≥ 1.24	Tableaux N-dimensionnels, algèbre linéaire, FFT
opencv-python	≥ 4.8	Traitement bas niveau, GrabCut, morphologie, Canny
scipy	≥ 1.11	Filtres, FFT, interpolation spline, matrices creuses
matplotlib	≥ 3.7	Visualisation, colourmaps, export PNG
Pillow (PIL)	≥ 10.0	Lecture/écriture d'images, conversion de formats

3.2 Bibliothèque optionnelle

TABLE 3.2 – Dépendance optionnelle (qualité IA)

Bibliothèque	Rôle
rembg	Interface Python pour le réseau U ² -Net (suppression de fond par IA)
onnxruntime	Moteur d'inférence ONNX (exécution du modèle neuronal)

3.3 Modules Python standard

TABLE 3.3 – Modules de la bibliothèque standard utilisés

Module	Usage
io	Tampons mémoire (BytesIO) pour l'export de fichiers
zipfile	Génération d'archives ZIP pour le traitement par lot

3.4 Sous-modules scipy utilisés

```

1 from scipy.ndimage      import gaussian_filter, convolve,
   median_filter
2 from scipy.interpolate  import RectBivariateSpline
3 from scipy.fft          import fft2, ifft2, fftshift, ifftshift
4 import scipy.sparse     as sc
5 from scipy.sparse.linalg import spsolve

```

Listing 3.1 – Imports scipy du projet

Chapitre 4

Fondements mathématiques

4.1 Représentation numérique d'une image

Une image en niveaux de gris est représentée comme une **matrice** $I \in [0, 1]^{H \times W}$, où chaque élément $I(i, j)$ représente l'intensité lumineuse du pixel (i, j) .

$$\hat{I}(i, j) = \frac{I(i, j) - \min(I)}{\max(I) - \min(I) + \varepsilon}, \quad \varepsilon = 10^{-8} \quad (4.1)$$

4.2 Convolution discrète 2D

$$(I * K)(i, j) = \sum_{s=-m}^m \sum_{t=-n}^n I(i-s, j-t) \cdot K(s, t) \quad (4.2)$$

Elle est utilisée pour le lissage gaussien, le calcul des gradients (Sobel) et le LoG.

4.3 Filtre Gaussien

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (4.3)$$

Le paramètre σ contrôle le niveau de lissage. `gaussian_filter()` de `scipy` applique ce filtre de manière **séparable** (deux passes 1D) pour l'efficacité.

4.4 CLAHE

L'égalisation adaptative d'histogramme avec limitation de contraste améliore le contraste local. Pour chaque tuile :

1. Calcul de l'histogramme local $h(v)$;

2. Écrêtage à `clipLimit`;
3. Redistribution de l'excès;
4. Application de la transformation cumulative :

$$T(v) = \frac{255}{N} \sum_{u=0}^v h(u) \quad (4.4)$$

```
clipLimit = 2.5      tileGridSize = (8, 8)
```

4.5 Filtre bilatéral

$$I_{\text{bil}}(i, j) = \frac{1}{W} \sum_{(k, l) \in \Omega} G_{\sigma_s}(i - k, j - l) \cdot G_{\sigma_c}(I(i, j) - I(k, l)) \cdot I(k, l) \quad (4.5)$$

- G_{σ_s} : poids gaussien **spatial** ($\sigma_s = \text{sigma_space}$);
- G_{σ_c} : poids gaussien **photométrique** ($\sigma_c = \text{sigma_color}$);
- W : facteur de normalisation.

Les pixels proches *et* de valeur similaire sont fortement pondérés $\Rightarrow$ les bords sont préservés.

4.6 Filtre médian

$$I_{\text{med}}(i, j) = \text{médiane}\{I(k, l) : (k, l) \in V(i, j)\} \quad (4.6)$$

Non-linéaire, très efficace contre le **bruit impulsif**. La taille du noyau est imposée impaire : `max(3, int(size)|1)`.

4.7 Interpolation par splines bicubiques

$$S(x, y) = \sum_i \sum_j c_{ij} \cdot B_i(x) \cdot B_j(y) \quad (4.7)$$

B_i sont les fonctions de base B-splines cubiques ; c_{ij} sont les coefficients calculés par moindres carrés. Le paramètre `scale` $\in [0.1, 1.0]$ contrôle le taux de sous-échantillonnage avant reconstruction.

4.8 Opérateur de Sobel et Non-Maximum Suppression

$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_y = K_x^T \quad (4.8)$$

$$M(i, j) = \sqrt{G_x^2 + G_y^2}, \quad \theta(i, j) = \arctan\left(\frac{G_y}{G_x}\right) \quad (4.9)$$

La **NMS** compare $M(i, j)$ à ses deux voisins dans la direction θ et ne conserve que les maxima locaux. Résultat : contours fins d'épaisseur **1 pixel**.

4.9 Algorithme de Canny

Étape 1. Lissage gaussien ;

Étape 2. Calcul du gradient (Sobel) ;

Étape 3. NMS ;

Étape 4. Double seuillage avec hystérèse (T_{bas} , T_{haut}).

`low` $\in [0.01, 0.30]$ `high` $\in [0.05, 0.60]$

4.10 Laplacien du Gaussien (LoG) – Zéro-croisements

$$\text{LoG} = \nabla^2(G_\sigma * I) = (\nabla^2 G_\sigma) * I \quad (4.10)$$

$$\nabla^2 G_\sigma(x, y) = \frac{1}{\pi\sigma^4} \left(\frac{x^2 + y^2}{2\sigma^2} - 1 \right) \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (4.11)$$

Noyau discret du Laplacien :

$$L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad (4.12)$$

$$\text{ZC}(i, j) = \mathbf{1}[\text{LoG}(i, j) \cdot \text{LoG}(i + \Delta i, j + \Delta j) < 0] \quad (4.13)$$

4.11 Transformée de Fourier 2D et filtres de Butterworth

$$F(u, v) = \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} I(i, j) e^{-2\pi i(ui/H + vj/W)} \quad (4.14)$$

$$H_{\text{PB}}(u, v) = \frac{1}{1 + D(u, v)^{2n}}, \quad H_{\text{PH}} = 1 - H_{\text{PB}} \quad (4.15)$$

$$D(u, v) = \sqrt{\left(\frac{u - H/2}{H \cdot f_c} \right)^2 + \left(\frac{v - W/2}{W \cdot f_c} \right)^2} \quad (4.16)$$

- ▷ **Passe-bas** : lisse l'image (atténue les hautes fréquences) ;
- ▷ **Passe-haut** : met en évidence les bords.

4.12 Saillance spectrale (Hou & Zhang, 2007)

1. $F = \mathcal{F}(I)$: FFT 2D ;
2. $A = \log(|F| + \varepsilon)$: log-amplitude spectrale ;
3. $R = A - \bar{A}_{3 \times 3}$: résidu spectral ;

4. $S = |\mathcal{F}^{-1}(e^{R+i\angle F})|^2$: carte de saillance.

Le résidu spectral R isole ce qui rend l'image unique par rapport à une image "moyenne". Les zones de forte saillance correspondent au **premier plan**.

4.13 GrabCut

GrabCut combine **GMM** et **MRF** en quatre étapes :

1. Initialisation du masque ;
2. Apprentissage des GMM de couleur ;
3. Segmentation par graphe (min-cut / max-flow) ;
4. Itérations alternant étapes 2 et 3.

Le masque est initialisé par **saillance spectrale** au lieu d'un rectangle fixe, ce qui améliore significativement la qualité.

4.14 Segmentation K-means en espace LAB

$$J = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \quad (4.17)$$

$$\mathbf{x}(i, j) = [L^*, a^*, b^*, 0.25 \cdot \tilde{i}, 0.25 \cdot \tilde{j}] \quad (4.18)$$

Le poids 0.25 limite l'influence spatiale. L'initialisation **K-means++** garantit une meilleure convergence.

4.15 Flood Fill depuis les bords

$$|I(x, y) - I(x_0, y_0)| \leq \text{tolerance} \quad (4.19)$$

Diffusion BFS depuis les 4 bords (espacement 3 pixels). Tout pixel non connecté aux bords est considéré comme **premier plan**.

4.16 Alpha matting par distance transform

$$\alpha(i, j) = \begin{cases} 1 & D_{\text{fg}}(i, j) \geq r \\ D_{\text{fg}}(i, j)/r & D_{\text{fg}}(i, j) < r \quad (\text{zone interne}) \\ 1 - D_{\text{bg}}(i, j)/r & D_{\text{bg}}(i, j) < r \quad (\text{zone externe}) \\ 0 & \text{sinon} \end{cases} \quad (4.20)$$

Transition douce de largeur r pixels entre sujet ($\alpha = 1$) et fond ($\alpha = 0$).

4.17 Transfert de couleur (Reinhard et al., 2001)

$$I'_c = \frac{I_c - \mu_s^c}{\sigma_s^c} \cdot \sigma_t^c + \mu_t^c \quad (4.21)$$

μ_s^c, σ_s^c : statistiques de la source ; μ_t^c, σ_t^c : statistiques de la cible.

4.18 Incrustation de Poisson (Pérez et al., 2003)

4.18.1 Formulation variationnelle

$$\min_{f^*} \iint_{\Omega} \|\nabla f^* - \mathbf{v}\|^2 dA \quad \text{s.c.} \quad f^*|_{\partial\Omega} = g|_{\partial\Omega} \quad (4.22)$$

La condition d'Euler-Lagrange donne :

$$\Delta f^* = \text{div}(\mathbf{v}) \quad \text{dans } \Omega, \quad f^* = g \quad \text{sur } \partial\Omega \quad (4.23)$$

Discrétisée en un système linéaire creux $A \mathbf{u} = \mathbf{b}$, résolu par `spsolve`.

4.18.2 Laplacien discret

$$\Delta_d f(i, j) = f(i + 1, j) + f(i - 1, j) + f(i, j + 1) + f(i, j - 1) - 4f(i, j) \quad (4.24)$$

4.18.3 Trois modes de fusion

TABLE 4.1 – Modes de fusion du Poisson Blending

Mode	Champ de guidage $\mathbf{v}$	Effet
Normal	∇f_s	Fusion classique
Gradient Max	$\arg \max(\ \nabla f_s\ , \ \nabla g\)$	Bords forts préservés
Gradient Min	$\arg \min(\ \nabla f_s\ , \ \nabla g\)$	Effet “fantôme”

4.18.4 Boîte englobante après rotation

$$W' = H|\sin \theta| + W|\cos \theta|, \quad H' = H|\cos \theta| + W|\sin \theta| \quad (4.25)$$

Chapitre 5

Module Pré-traitement & Lissage

5.1 `normalize()` / `to_uint8()`

Fonctions de conversion entre `float64` $\in [0, 1]$ et `uint8` $\in [0, 255]$, utilisées avant chaque appel OpenCV.

5.2 `clahe_enhance(img)`

- ▷ **Entrée** : image `float64` $[0, 1]$
- ▷ **Sortie** : image `float64` $[0, 1]$, contraste local amélioré
- ▷ **Effet** : révèle les détails fins dans les zones sombres ou surexposées

5.3 `gaussian_smoothing(img, sigma=1.5)`

Pré-traitement standard avant Sobel, Canny et LoG. $\sigma \in [0.3, 6.0]$.

5.4 `bilateral_smoothing(img, ...)`

Lissage **préservant les bords** — idéal avant segmentation. Opération non-linéaire, plus coûteuse que le gaussien.

5.5 `median_smoothing(img, size=5)`

Élimine le bruit impulsionnel sans flouter les bords. Taille imposée impaire.

5.6 `spline_smooth(img, scale=0.5)`

Sous-échantillonnage, calcul de la spline bicubique, réévaluation à la résolution originale. Lissage global par représentation réduite.

Chapitre 6

Module Détection de Contours

6.1 `sobel_nms(img)`

Implémentation **manuelle** Sobel + NMS en Python pur. Complexité : $\mathcal{O}(H \times W)$.

6.2 `canny_edges(img, low, high)`

Délègue à `cv2.Canny()` (C++ optimisé) avec `L2gradient=True`.

6.3 `log_zero_crossing(img, sigma)`

Détecte les passages de signe du LoG dans les directions horizontale et verticale. Robuste au bruit grâce au pré-lissage intégré.

6.4 `spline_edges(img, scale)`

Gradient par `np.gradient()` (différences finies centrées d'ordre 2) sur l'image splinée.

6.5 `fft_hp / fft_lp(img, cutoff, order)`

Pipeline complet :

$$I \xrightarrow{\text{FFT}} F \xrightarrow{\text{fftshift}} \tilde{F} \xrightarrow{\times H} \tilde{F}H \xrightarrow{\text{ifftshift}} FH \xrightarrow{\text{IFFT}} |\cdot| \xrightarrow{\text{norm.}} \text{résultat}$$

6.6 `fft_spectrum(img)`

Retourne $\log(1 + |F|)$ pour compresser la dynamique du spectre.

Chapitre 7

Module Suppression d'Arrière-plan

7.1 `spectral_saliency(img_gray)`

Pré-calcul utilisé comme initialisation pour GrabCut et comme diagnostic visuel. Isole les anomalies fréquentielles correspondant aux régions saillantes.

7.2 `bg_remove_grabcut(...)`

Améliorations par rapport au GrabCut standard :

- ▷ Initialisation par masque de saillance ;
- ▷ Double seuillage Otsu (`GC_PR_FGD` vs `GC_FGD`) ;
- ▷ Repli automatique sur GrabCut rectangulaire en cas d'échec ;
- ▷ Marge ajustable en pourcentage de la plus petite dimension.

7.3 `bg_remove_border_flood(...)`

Points de départ espacés de 3 pixels le long des 4 bords. Travaille sur l'image en niveaux de gris floutée pour la robustesse aux textures fines.

7.4 `bg_remove_kmeans(img_color, k)`

Vote par les bords (6 pixels intérieurs). Pour $k = 3$, le 2^e cluster le plus présent aux bords est aussi fond, sauf s'il occupe plus de 30 % du centre.

7.5 `bg_remove_rembg(...)`

Interface vers **rembg** (U²-Net). Modèles disponibles : de `u2netp` (léger) à `birefnet-massive` (haute qualité). `@st.cache_resource` évite le rechargement.

Chapitre 8

Module Raffinement du Masque

8.1 `fill_holes(mask)`

Composantes connexes du masque inversé ne touchant pas les bords $\Rightarrow$ trous $\Rightarrow$ remplissage.

8.2 `remove_islands(mask, min_ratio)`

Composantes d'aire $< \text{min_ratio} \times H \times W$ supprimées comme artefacts.

8.3 `edge_snap(mask, img_gray, snap_radius)`

1. Contours Canny;
2. Dilatation pour créer une zone d'attraction;
3. Érosion légère du masque;
4. Sur la bordure : contour proche $\Rightarrow$ masque érodé, sinon $\Rightarrow$ masque original.

8.4 `alpha_matting(mask, radius)`

`cv2.distanceTransform` fournit les distances internes/externes; rampe linéaire de largeur `radius` (équation 4.20).

8.5 `full_refinement(...)`

Pipeline séquentiel :

1. Fermeture morphologique (noyau elliptique 5×5);
2. `fill_holes()`;
3. `remove_islands()`;
4. `edge_snap()`;
5. `alpha_matting()` $\rightarrow$ masque binaire affiné et alpha continu.

Chapitre 9

Module Composition & Poisson Blending

9.1 `get_poisson_operators(m, n)` — `cache`

Construit trois matrices creuses CSC de taille $mn \times mn$: D_x (diff. horizontale), D_y (diff. verticale), A (Laplacien 5 diagonales). Calculées une seule fois par taille grâce à `@st.cache_data`.

9.2 `make_S_sparse(mask)`

Matrice de sélection $S \in \{0, 1\}^{N_\Omega \times mn}$ (format CSR). Chaque ligne isole un pixel de Ω .

9.3 `poisson_blend_patch(...)`

```
1 # 1. Calculer le second membre b (selon le mode)
2 # 2. Reduire au domaine Omega
3 A_final = S @ A @ S.T # symetrique definie positive
4 # 3. Resoudre par solveur direct creux
5 u = spsolve(A_final, b)
6 # 4. Reconstruire l'image complete
```

Listing 9.1 – Schéma de résolution du système de Poisson

9.4 `apply_reflection(...)`

$$\text{opacité}(i) = \text{opacity} \times \left(1 - \frac{i}{\text{reflection_h}}\right) \quad (9.1)$$

Chapitre 10

Fonctions Utilitaires & Post-traitement

10.1 `color_transfer(source, target)`

LAB $\rightarrow$ statistiques $\rightarrow$ transformation affine (éq. 4.21) $\rightarrow$ RGB.

10.2 `apply_mask_soft(img, alpha, bg)`

$$\text{out}(i, j) = \alpha(i, j) \cdot I(i, j) + (1 - \alpha(i, j)) \cdot \text{bg} \quad (10.1)$$

10.3 `apply_image_adjustments(...)`

- ▷ **Luminosité / Contraste** : $\alpha_c = 1 + c/100$, $\beta = b$ via `cv2.convertScaleAbs` ;
- ▷ **Saturation** : canal S en espace HSV ;
- ▷ **Neteté** (Unsharp Masking) : $I_{\text{sharp}} = (1 + s) \cdot I - s \cdot G_3(I)$.

10.4 `apply_artistic_mode(...)`

- ▷ **Sketch** : `gray/(255 - blur(255 - gray))` ;
- ▷ **Blueprint** : Canny + fond bleu profond + grille.

10.5 `generate_shadow(...)`

Translation de l'alpha + flou gaussien + multiplication par `opacity`.

10.6 `apply_watermark(...)`

Double tracé (contour + texte) via `cv2.putText`, fusion par `cv2.addWeighted`.

Chapitre 11

Interface Streamlit

11.1 Mise en cache

```
1 @st.cache_data      # cached_contours, cached_bg,  
    get_poisson_operators  
2 @st.cache_resource  # get_rembg_session (modele IA)
```

Listing 11.1 – Décorateurs de mise en cache

`@st.cache_data` sérialise les arguments (bytes + tuple de paramètres) comme clé. `@st.cache_resource` conserve une instance unique du modèle IA entre les sessions.

11.2 Organisation en onglets

TABLE 11.1 – Onglets de l’interface Streamlit

Onglet	Contenu
Vue unique	Sélection d’une méthode, affichage, export PNG
Vue complète	Grille $4 \times N$ de toutes les méthodes, export
Comparaison	Côte-à-côte + carte de différence
Suppression Fond	Pipeline + diagnostics + exports
Composition Poisson	Incrustation interactive avec aperçu live
Avant/Après	Comparaison visuelle

11.3 Traitement par lot

Boucle sur tous les fichiers, même segmentation + filigrane, archive générée en mémoire via `io.BytesIO` + `zipfile`.

Chapitre 12

Flux de données (Pipeline)

Pipeline général de traitement

```
Image chargée (bytes)
|
v
[Decodage OpenCV] --> raw_gray (float64) + raw_rgb (uint8)
|
+-----+
v                                     v
[cached_contours()]                 [cached_bg()]
|                                     |
v                                     v
CLAHE -> Gaussien                     Choix methode segmentation
|                                     GrabCut / FloodFill /
+- Bilateral                         K-means / rembg
+- Median                             |
+- Spline                             v
+- Sobel+NMS                         raw_mask (binaire)
+- Canny                             |
+- LoG                               v
+- FFT Spectre                       full_refinement()
+- FFT PH                             |   |
+- FFT PB                           refined_mask alpha
|                                     |
v                                     v
                                apply_mask_soft() --> resultats
|
+-----+-----+
v             v             v
Filtre artistique   Reflet   Ajustements
|                   |       |
+-----+-----+
|
```

```
Watermark
|
Export PNG/ZIP
|
[Poisson Blending optionnel]
transform_patch()
poisson_blend_patch()
```

Chapitre 13

Tableau récapitulatif des méthodes

TABLE 13.1 – Comparaison des méthodes implémentées

Méthode	Famille	Complexité	Qualité bords	Paramètres
Gaussian	Lissage linéaire	$O(HW\sigma)$	×	σ
Bilatéral	Lissage non-lin.	$O(HWd^2)$	✓✓	σ_c, σ_s
Médian	Lissage non-lin.	$O(HWk^2)$	✓	k
Spline	Interpolation	$O(HW)$	✓	scale
Sobel + NMS	Gradient 1 ^{er} ordre	$O(HW)$	✓	—
Canny	Multi-étapes	$O(HW)$	✓✓✓	T_l, T_h
LoG	Gradient 2 ^e ordre	$O(HW)$	✓✓	σ
FFT Butterworth	Fréquentiel	$O(HW \log HW)$	✓✓	cutoff, order
GrabCut	Probabiliste/MRF	$O(HW \cdot \text{iter})$	✓✓	iter, margin
Flood Fill	Topologique	$O(HW)$	✓	tolerance
K-means LAB	Clustering	$O(HWk \cdot \text{iter})$	✓	k
rembg U ² -Net	Deep Learning	$O(HW)_{\text{GPU}}$	✓✓✓✓	model
Poisson Blending	EDP / Alg. lin.	$O(N_{\Omega}^{1.5})$	✓✓✓✓	mode, pos

Références bibliographiques

- [1] **Canny, J.** (1986). *A Computational Approach to Edge Detection*. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), 679–698.
- [2] **Hou, X. & Zhang, L.** (2007). *Saliency Detection : A Spectral Residual Approach*. In *Proceedings of CVPR*.
- [3] **Pérez, P., Gangnet, M., & Blake, A.** (2003). *Poisson Image Editing*. In *ACM SIGGRAPH*.
- [4] **Rother, C., Kolmogorov, V., & Blake, A.** (2004). “GrabCut” — *Interactive Foreground Extraction using Iterated Graph Cuts*. In *ACM SIGGRAPH*.
- [5] **Reinhard, E., Adhikhmin, M., Gooch, B., & Shirley, P.** (2001). *Color Transfer between Images*. *IEEE Computer Graphics and Applications*, 21(5), 34–41.
- [6] **Tomasi, C. & Manduchi, R.** (1998). *Bilateral Filtering for Gray and Color Images*. In *Proceedings of ICCV*.
- [7] **Qin, X., Zhang, Z., Huang, C., Dehghan, M., & Jagersand, M.** (2020). *U²-Net : Going Deeper with Nested U-Structure for Salient Object Detection*. *Pattern Recognition*, 106.
- [8] **Pizer, S., Amburn, E., Austin, J., et al.** (1987). *Adaptive Histogram Equalization and Its Variations*. *Computer Vision, Graphics, and Image Processing*, 39(3), 355–368.